Informe de Proyecto

Programación con struct y fstream: Mini Juego «Piedra, Papel o Tijera» con Persistencia de Datos — Actividad 7

EEST N°1 de Vicente López

Laboratorio de Programación (LPR) — 5° 3° A-B

Alumno: Lucas Del Pino

Curso: 5° 3° A-B

Docente: Prof. Mansilla Muñoz York, Prof. Yamil De Jesus Ganduglia

Repositorio GitHub: https://github.com/Pin0l33/piedrapapelotijera.git

Repositorio GitLab: https://gitlab.com/Pin0l33/piedrapapelotijera.git

Introducción

El presente informe documenta la Actividad 7 de la materia Laboratorio de Programación (LPR), correspondiente al 2° cuatrimestre de 2026. El objetivo fue comprender la diferencia entre la memoria RAM (volátil) y el almacenamiento secundario (no volátil) mediante el desarrollo de un mini juego de Piedra, Papel o Tijera contra la inteligencia artificial de la consola. Cada partida se modela con una estructura (struct) llamada Partida, que agrupa el ID, el jugador, la elección del jugador, la elección de la IA y el resultado, y se guarda de forma permanente en un archivo de texto (historial_partidas.txt) mediante la librería <fstream>. Además, se aplicó la metodología 5R (Registrar, Reducir, Responder, Reflexionar, Revisar) y se organizó el trabajo en un repositorio con sincronización dual a GitHub y GitLab.

Desarrollo

Registro del código y ejecución (R1)

El código fuente se guardó en src/main.cpp. Se compila con `g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe` y se ejecuta con `.\src\piedrapapelotijera.exe` en PowerShell (Windows 10/11, VS Code). El programa muestra un menú con tres opciones: (1) jugar una partida, (2) ver el historial guardado en disco y (3) salir. Al jugar, el usuario ingresa su nombre y elige Piedra, Papel o Tijera; la IA elige al azar con rand() % 3 + 1 y el programa determina Victoria, Derrota o Empate.

Figura 1

Menú principal y partida jugada en la consola

```
==================================================
  LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA
  Estudiante / Grupo: Lucas Del Pino - 5 3 A-B
==================================================

--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---
1. Jugar Partida (Piedra, Papel o Tijera)
2. Ver Historial de Partidas Guardadas en Disco (.txt)
3. Salir del Sistema
Ingrese una opcion (1-3): 1

--- NUEVA PARTIDA #1 ---
Ingrese su Nombre / Nickname: lucas

Elija su jugada:
1. Piedra
2. Papel
3. Tijera
Opcion (1-3): 2

[JUGADA] lucas eligio: Papel
[JUGADA] La IA eligio: Papel
=> RESULTADO: Empate!
[DISCO] Partida resguardada con exito en 'historial_partidas.txt'.
```

Núcleo del programa (R2)

Las líneas clave del código son las que abren los flujos de archivo:

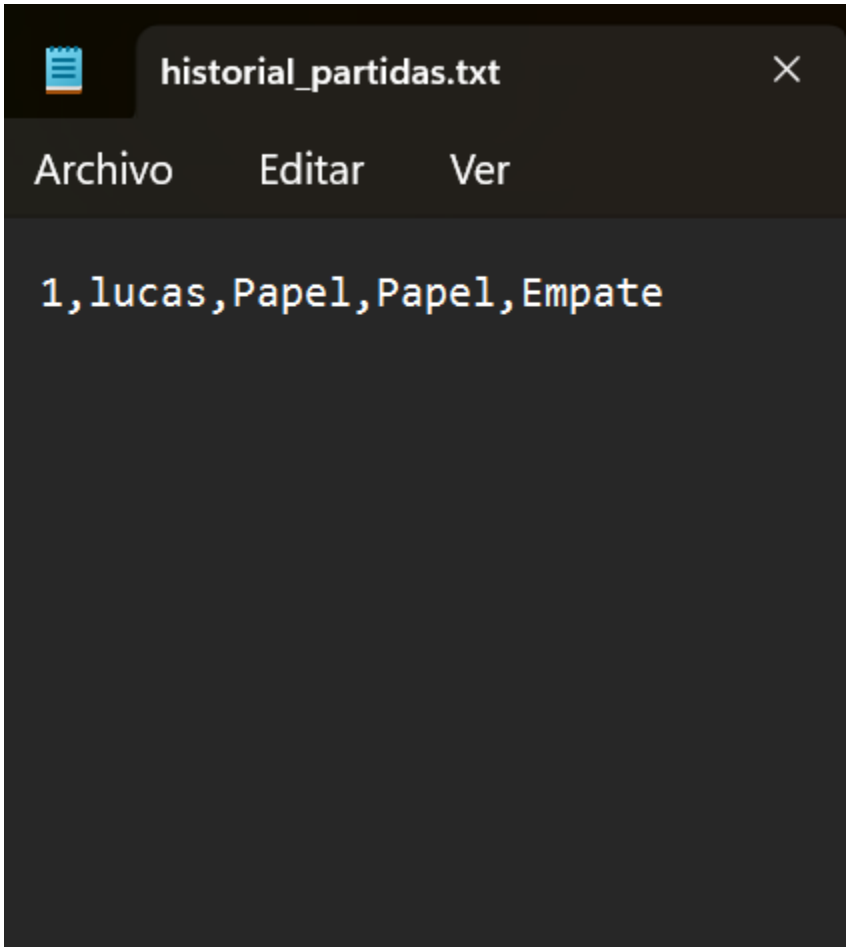
- `ofstream archivo("src/historial_partidas.txt", ios::app);` abre el archivo para escritura en modo Append, de modo que cada partida nueva se agrega al final sin borrar las anteriores.
- `ifstream archivo("src/historial_partidas.txt");` abre el archivo para lectura y permite recuperar todo el historial con getline() separando los campos por comas.

La estructura Partida agrupa cinco datos heterogéneos: idPartida (int) y jugador, eleccionJugador, eleccionIA y resultado (string). Cada registro se guarda en una línea en formato CSV: `ID,Jugador,EleccionJugador,EleccionIA,Resultado`. El programa se divide en tres funciones modulares: jugarPartida(), guardarEnDisco() y leerHistorialDisco().

Demostración de persistencia

Para verificar la persistencia se jugó una partida, se cerró la consola, se volvió a abrir el ejecutable y se eligió la Opción 2, que mostró las partidas guardadas antes del cierre.

Figura 2



Repositorio y sincronización Dual-Remote (R5)

El proyecto se organizó en el repositorio piedrapapelotijera con las carpetas src/, docs/, data/ y capturas/, además de los archivos README.md, LICENSE, CHANGELOG.md, .gitignore y .gitattributes. El código y las capturas se subieron a GitHub y GitLab con una secuencia de push dual:

```
git add .
git commit -m "Actividad 7: piedra, papel o tijera con persistencia v1.0.0"
git push origin main
git push gitlab main
```

Tabla 1

Repositorios remotos del proyecto

Plataforma	URL
GitHub	https://github.com/Pin0l33/piedrapapelotijera.git
GitLab	https://gitlab.com/Pin0l33/piedrapapelotijera.git

Observaciones técnicas

Durante las pruebas se observaron dos limitaciones del código base. Primero, contadorID se inicializa en 1 cada vez que se abre el programa, por lo que los IDs se repiten entre sesiones aunque el archivo conserve las partidas anteriores. Segundo, cualquier opción distinta de 1 o 2 en la jugada se interpreta como Tijera, ya que no hay validación de entrada. Ambas son mejoras posibles para una versión futura.

Cuestionario técnico de control

Pregunta 1: RAM vs. Disco

¿Por qué si cerrás la ventana de consola y volvés a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?

Porque las variables del programa viven en la memoria RAM, que es volátil: cuando el programa termina, esos datos se pierden. En cambio, cada partida se escribe con ofstream en el archivo historial_partidas.txt, que está en el almacenamiento secundario (disco rígido o de estado sólido), y este es no volátil, por lo que sus datos permanecen aunque el programa se cierre. Al volver a abrir el ejecutable y elegir la Opción 2, ifstream lee ese archivo y recupera todas las partidas guardadas.

Pregunta 2: Uso de ios::app

¿Qué sucedería con el archivo historial_partidas.txt si en la función guardarEnDisco utilizaras ofstream archivo("historial_partidas.txt"); sin incluir la opción ios::app?

Por defecto, ofstream abre el archivo en modo de escritura (ios::out), que trunca el contenido existente al abrirlo. Entonces, cada vez que se guardara una partida, el archivo se vaciaría y se escribiría solo la nueva, de modo que el historial quedaría siempre con un único registro (la última partida) y se perderían todas las anteriores. Con ios::app, los datos nuevos se agregan al final y se conserva lo que ya estaba.

Pregunta 3: Manejo del búfer (cin.ignore())

¿Por qué es necesario invocar cin.ignore() antes de leer el nombre del jugador con getline(cin, p.jugador)?

Cuando el usuario elige una opción del menú con `cin >> opcion`, escribe el número y presiona Enter. El operador >> lee el número, pero deja el carácter de salto de línea (\n) pendiente en el búfer de entrada. Si se llamara directamente a getline(), este leería ese \n sobrante como una línea vacía y la pregunta del nombre se saltearía. cin.ignore() descarta ese carácter pendiente y deja el búfer limpio para que getline() espere el nombre real.

Pregunta 4: Agrupación con struct

¿Qué ventajas ofrece la estructura Partida para organizar las variables del juego frente a pasar 5 variables sueltas a cada función?

La estructura Partida agrupa en una sola entidad los cinco datos de una partida, lo que ofrece varias ventajas: las funciones reciben un único parámetro (por ejemplo, `guardarEnDisco(const Partida& p)`) en lugar de cinco, lo que reduce errores de orden o de tipo; el código es más legible porque los datos se acceden por nombre (p.resultado, p.jugador); si hay que agregar un campo nuevo, se modifica solo la definición del struct y no la firma de cada función; y el paso por referencia constante evita copiar los datos y protege la partida de modificaciones accidentales.

Pregunta 5: Aplicación al P.I.A.

Si adaptaran este módulo de almacenamiento a su Proyecto Integrador Anual, ¿qué datos o eventos de su prototipo guardarían en el archivo de texto?

Nuestro Proyecto Integrador Anual es un juego RPG hecho en Unity, y la misma idea de este módulo se aplica ahí: la estructura Partida se reemplaza por un registro propio del juego (en Unity, una clase o struct marcada como serializable) que se guarda en un archivo del disco. Se guardarían dos tipos de datos. Primero, la partida guardada del jugador: nombre del personaje, nivel, experiencia, vida actuales, posición y escena en la que está, inventario y oro, y misiones completadas, para que el progreso sobreviva al cierre del juego (la RAM es volátil, igual que en esta actividad). Segundo, un historial de eventos, en el que se agrega una línea por cada hecho importante sin borrar las anteriores: enemigos derrotados, ítems obtenidos, muertes del jugador, misiones aceptadas o cumplidas y cada jefe vencido, con su fecha y hora. También se podrían guardar las configuraciones (volumen, controles, resolución). El equivalente de ofstream con ios::app en Unity es escribir con un StreamWriter en modo append, y el de ifstream es leer con un StreamReader, o con JsonUtility si se usa formato JSON, siempre dentro de la carpeta Application.persistentDataPath.

Conclusión

La actividad permitió comprender la diferencia entre la memoria RAM, donde los datos se pierden al cerrar el programa, y el almacenamiento en disco, donde se conservan. El uso de un struct para modelar la partida organizó mejor los datos y simplificó las funciones, mientras que ofstream con ios::app y ifstream permitieron guardar y recuperar el historial de forma persistente. También se comprobó la importancia de cin.ignore() para evitar errores con el búfer de entrada. Por último, la sincronización dual con GitHub y GitLab y la documentación en formato APA v7 reforzaron las buenas prácticas de versionado y de documentación del software.